Question 1: Bias-Variance Trade-off Simulation

Part 1. Simulation Results

We conducted 100 simulation repetitions using random seed **42**. As node size increases from 1 to 30, squared bias increases substantially, while variance decreases slightly. This shows that larger node sizes make the trees less flexible, leading to higher bias but lower variance.

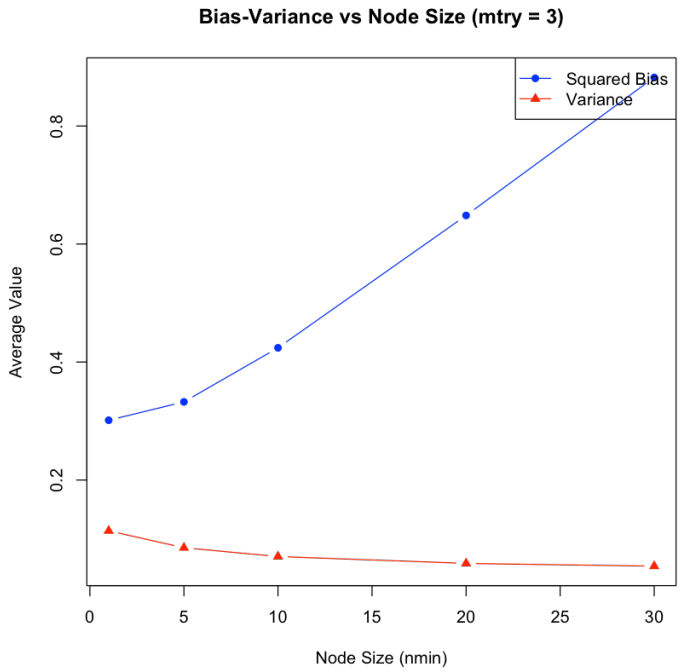


Figure 1: Effect of Node Size (mtry = 3)

As mtry increases from 1 to 5, squared bias decreases substantially, while variance increases slightly. A larger mtry allows the trees to consider more important predictors at each split, improving the fit but increasing variance.

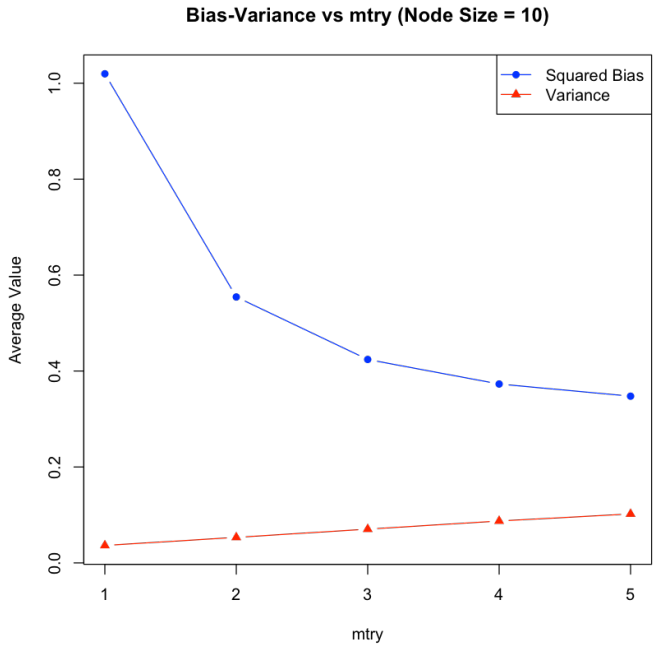


Figure 2: Effect of mtry (Node Size = 10)

Part 2. Discussion

1. How does a large node size, such as 30, affect the fit?

A large node size leads to shallower trees and less flexible predictions. In our simulation, node size = 30 produces high squared bias and low variance, suggesting underfitting.

2. What are the benefits and drawbacks of using a large mtry?

A larger mtry increases the chance of selecting important predictors (X1, X3, and X5), reducing squared bias. However, it can make trees more correlated and increase the variance of the forest's predictions. Our simulation shows this trade-off.

3. What if each tree samples 100 instead of 200 observations?

I would expect the preferred node size to shift toward smaller values. With fewer observations per tree, smaller node sizes allow deeper splits and may help reduce bias. However, they can also increase variance, so the preferred setting is not guaranteed to change in this direction.

Question 2: Tuning Random Forests for MNIST

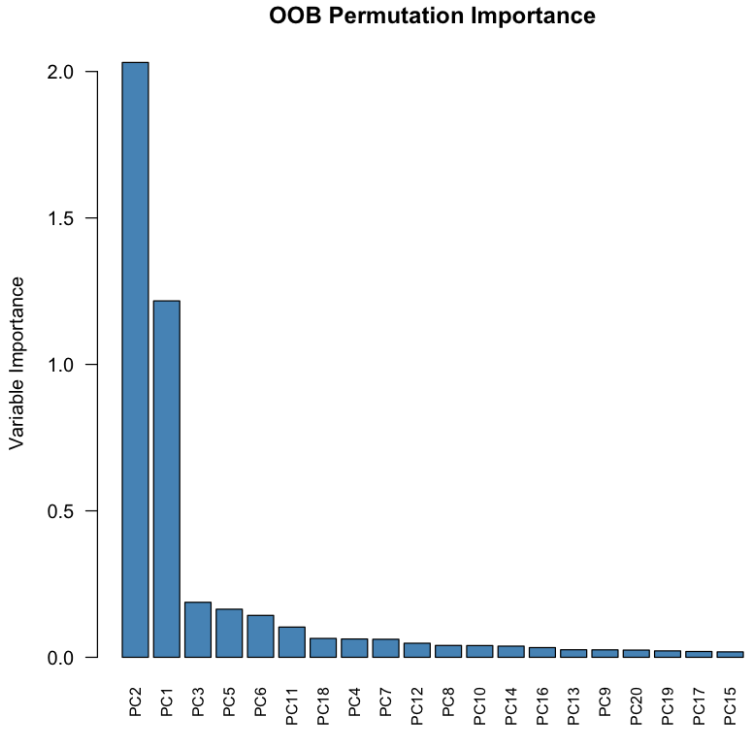
Part 1. Default Random Forest

We retained digits 1, 6, and 7 and applied PCA using the training data only, keeping the first 20 principal components. A default RLT classification forest with 1,000 trees achieved an OOB misclassification rate of **21.43%**.

Part 2. Hyperparameter Tuning and Variable Importance

We evaluated 25 combinations of mtry and node size using OOB misclassification rate. The selected combination was **mtry = 8 and node size = 5**, with the lowest reported OOB error of **6.50%**.

Permutation importance showed that PC2 (2.0308) and PC1 (1.2169) were the most influential predictors, followed by PC3 (0.1877). This suggests that the first two principal components contain much of the information used by the forest to distinguish the three digits.



PC2 and PC1 have the highest permutation importance, suggesting that these two principal components are particularly useful for distinguishing digits 1, 6, and 7. The remaining components contribute substantially less.

Part 3. Final Test Evaluation

The selected forest achieved a test misclassification rate of **1.58%**, corresponding to **98.42% accuracy**.

The confusion matrix shows that 312 of 317 test observations were classified correctly. Most errors involved confusing digit 7 with digit 6.

Overall, the tuned random forest performed well on the independent test data.

Table 2. Test Confusion Matrix

Actual / Predicted	1	6	7
1	103	0	1
6	1	105	0
7	0	3	104

Question 3: Planning a Random-Forest Analysis Skill

Part 1. Skill Design Philosophy

I would write a short and flexible skill that focuses on the purpose of each step rather than giving too many detailed instructions. If the AI is capable, I would let it handle standard tasks such as data preprocessing and model fitting, while providing clear requirements for tuning, evaluation, and reporting. For a less capable AI, I would include more detailed guidance to avoid mistakes. My personal preferences would focus on comparing models fairly, avoiding data leakage, using clear tables and figures, and explaining why the final model was selected. The goal is to make the analysis reliable, easy to understand, and reproducible without unnecessary instructions.

Part 2. Ten Instructions (Basic → Personal Preferences)

1. **Check the data:** Identify the response, predictors, sample size, variable types, missing values, and class distribution if applicable.
2. **Prepare the data:** Handle missing values and categorical predictors appropriately while avoiding data leakage.
3. **Choose the task:** Determine whether to use regression or classification random forests.
4. **Fit a baseline:** Train a default random forest before tuning to establish a reference.
5. **Tune hyperparameters:** Compare reasonable values of `mtry` and node size, and consider other parameters when necessary.
6. **Evaluate fairly:** Use OOB error or cross-validation to compare candidate models, keeping the final test data separate.
7. **Select the model:** Prefer the model with the lowest validation error, while considering stability and model complexity.
8. **Present a table:** Report candidate settings and their errors, sorted from lowest to highest error.

9. **Create figures:** Show model performance across tuning settings and permutation variable importance in clear, compact figures.
10. **Interpret and report:** Explain the selected model, important predictors, key numerical results, and limitations in a concise report that prioritizes meaningful findings over unnecessary technical details.